

KLITE RPmod — User Guide

KLITE RPmod adds roleplay tooling to **Esobold Esolite** (a KoboldAI Lite fork): character panels, a guided "Easy mode" for beginners, and the **Worlds** system — a living, graph-based world your story can move through. This guide focuses on using it, with an emphasis on the **in-chat Worlds commands**.

1. Installing

The mod ships as a single file, `KLITE-RPmod.js`.

1. In Esolite, open the **mod manager** (settings → user mod).
2. Load / paste `KLITE-RPmod.js` and enable it. Reload when prompted.
3. After it loads you'll see the existing RPmod panels, plus a  **Worlds** button in the navbar and a small **Worlds panel** in the bottom-right corner.

That's it — one file contains everything (main panels, guided onboarding, the Worlds engine, and the Worlds editor).

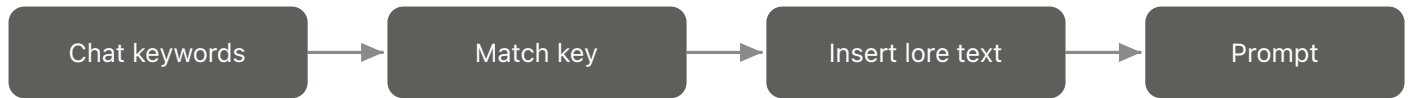
2. What you get

- **Right-side panels** — CHARS, ROLES, TOOLS, CONTEXT, IMAGES: manage characters, personas and groups, context tools, and image generation.
- **Guided RP** — a beginner-friendly step-by-step setup overlay (toggleable with an Advanced mode for the full interface).
- **Worlds** — the focus of this guide, below.

3. What is Worlds?

A classic lorebook dumps in any entry whose keyword appears in the chat — with no sense of *where* or *when* you are. Worlds instead models a real world (locations, NPCs, factions, objects, events, a clock) and feeds the AI **only the slice relevant to your current location and time**.

Classic lorebook — keyword → text



Every matched entry is dumped in, no sense of place or time.

Worlds — state → relevant slice



Only the current location's characters, objects, events and relevant lore — not the whole world.

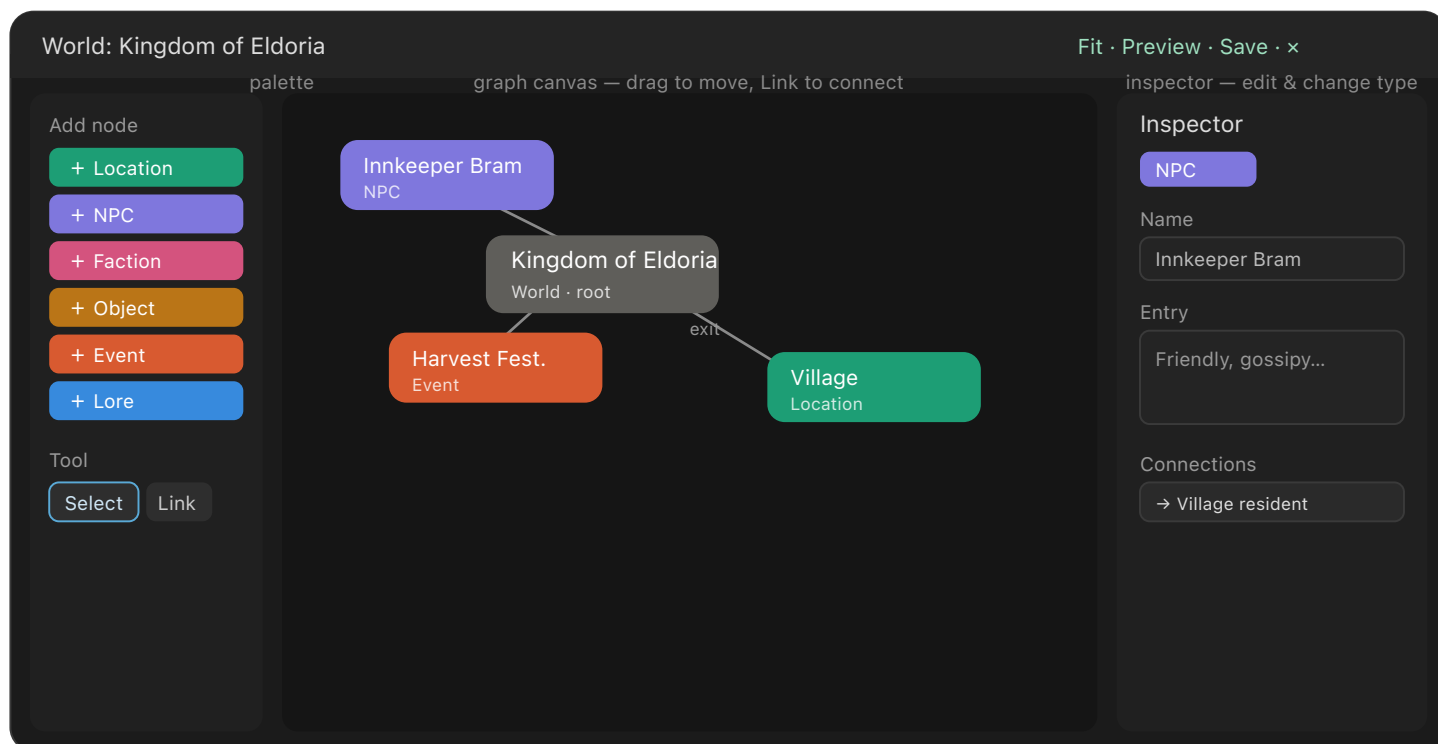
Result: less prompt bloat, better consistency, real location & time awareness.

The result: less prompt bloat, better consistency, and genuine location & time awareness. Persistent NPCs move on schedules, events fire when their conditions are met, and the world remembers state (flags, inventory, quest progress) as you play.

Under the hood the mod hands Esolite a compact, per-turn summary through its normal WorldInfo pipeline — you don't have to manage any of that; it's automatic.

4. Building a world (the editor)

Open the editor with the  **navbar button** or **Open editor** in the Worlds panel.
(No world yet? It'll offer to create one.)



- **Palette (left)** — click a type to drop a colour-coded node:
Location · NPC · Faction · Object · Event · Lore.
- **Canvas (middle)** — drag nodes to arrange them. Wheel to zoom, drag empty space to pan, **Fit** to reframe.
- **Tools** — **Select** (move nodes) and **Link** (click one node, then another, to connect them).
- **Inspector (right)** — edit the selected node's name, entry, and type-specific fields; review/remove its connections; change its **type**; or delete it.
- **Save** writes the world to your library; **Preview** shows exactly *what the AI will* see for your current location.

Connections carry meaning

You don't label arrows — the editor infers the relationship from the two node types and writes it into the data:

Drag between	Becomes
Location ↔ Location	an exit (both directions)
NPC → Location	NPC lives / is stationed there
NPC → Faction	NPC belongs to that faction
Object → Location	object is located there
Object → NPC	object is owned by that NPC
Event → Location	event can occur there

So the graph you draw *is* what the AI traverses. Invalid pairs (e.g. NPC → NPC) are rejected.

Tip: after **importing** a lorebook (section 8) everything arrives as grey **Lore** nodes. Select one and use the inspector's **type** dropdown to promote it to a Location or NPC, then wire up exits.

5. Playing with a world

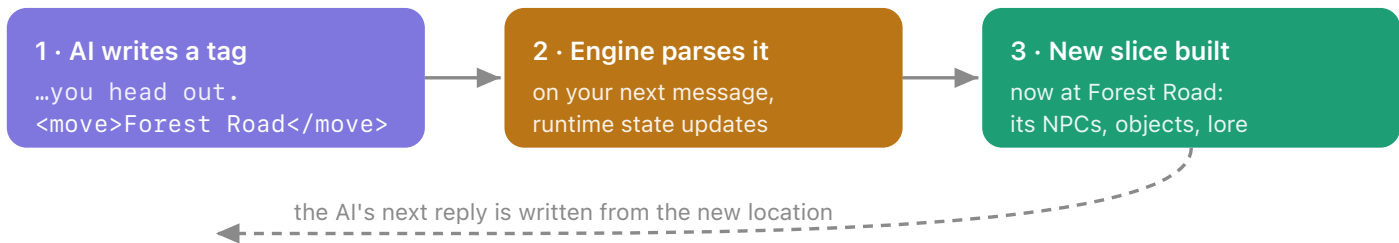
From the **Worlds panel** (bottom-right):

- Pick a world from the dropdown (or **+ New**).
- **Enable for this story** — turns the world on for the current chat. (Off = Esolite behaves normally; nothing is injected.)
- **Current location** — set where the player is; the slice follows it.
- **Clock** — shows day / time / season, with **▶▶** to advance time.

Now just chat. Each turn, the AI receives your current location, the NPCs and objects there, active events, and relevant lore — and nothing from the far side of the map.

6. In-chat commands (Worlds tags)

The world changes as you play through small **tags**. The AI can emit them in its replies (instruct it to, via your World Rules), or **you can type them yourself**. They're parsed on your next turn, update the world state, and are reflected in the following slice.



Categories

- movement `<move>` `<npcmove>`
- characters `<mood>`
- state `<flag>` `<quest>` items
- time `<time>` `<weather>` `<advance>`
- inventory `<give>` `<take>`

Tip: you can type these yourself too — e.g. send `<move>Tavern</move>` to relocate.

Command reference

Command	Example	Effect
<code><move>...</move></code>	<code><move>Forest Road</move></code>	Move the player to a location
<code><npcmove>NPC=Loc</npcmove></code>	<code><npcmove>Captain Rowan=Village</npcmove></code>	Move an NPC to a location
<code><mood>NPC=Mood</mood></code>	<code><mood>Bram=cheerful</mood></code>	Set an NPC's mood
<code><flag>key=value</flag></code>	<code><flag>metRowan=true</flag></code>	Set a story flag (<code><flag>key</flag></code> = true)
<code><unflag>key</unflag></code>	<code><unflag>metRowan</unflag></code>	Clear a flag
<code><give>Item</give></code>	<code><give>Torch x2</give></code>	Add to inventory (x2 optional)
<code><take>Item</take></code>	<code><take>Torch</take></code>	Remove from inventory
<code><quest>id=state</quest></code>	<code><quest>find_sword=active</quest></code>	Set a quest's state (done hides it)
<code><time>slot</time></code>	<code><time>evening</time></code>	Set time of day
<code><weather>...</weather></code>	<code><weather>rain</weather></code>	Set the weather
<code><advance></code>	<code><advance></code>	Advance the clock one step

Time slots: morning → noon → afternoon → evening → night . Advancing past night rolls

to the next day, and the **season** follows the month automatically.

Notes

- Tags are applied at the **start of your next message**, so a move the AI narrates this turn takes effect from the next turn onward — which reads naturally in the story.
- Tags currently remain visible in the chat text (they're read, not hidden). To have the AI use them, add a line to your **World Rules** such as: *"When the scene changes location, emit <move>Name</move> ."*
- Want time to pass automatically every turn? Enable it once in the console:
`KLITE_RPMod_Worlds.config.advanceClockPerTurn = true .`

7. What the AI actually sees

Click **Preview** in the editor (or run `KLITE_RPMod_Worlds.preview()` in the console) to see the assembled slice, e.g.:

[World Rules]

Medieval fantasy tone.

[Current Time]

Day 1, spring (morning). Weather: clear.

[Current Location: Village]

A small farming village.

Exits: Forest Road

[Nearby NPCs]

– Innkeeper Bram | friendly, gossipy

[Relevant Lore]

The King rules from the distant capital.

Sections are prioritised (rules, time, player state, location, NPCs, objects, events, lore) and trimmed to fit your context budget, so the important things survive.

8. Import & export

In the **Worlds** panel:

- **↓ Import** — load a classic WorldInfo / lorebook / TavernCard `character_book` JSON. Entries become **Lore** nodes you can promote to Locations/NPCs in the editor. You'll be asked whether to merge into the active world or create a new one.

asked whether to merge into the active world or create a new one.

- **↑ Export** — save the world as portable **World JSON**, or down-convert to a flat **WorldInfo** file compatible with vanilla Lite.

Worlds are stored in your browser (IndexedDB); the per-story runtime state (where you are, flags, inventory, clock) is saved inside the story savefile automatically. Your saved stories never contain the temporary injected entries.

9. Power-user console commands

Everything is scriptable via `KLITE_RPMod_Worlds` (alias `W` below):

```
const W = KLITE_RPMod_Worlds;
W.newWorld('Eldoria');           // create + activate a world
W.openEditor();                  // open the graph editor
W.enable(); W.disable();         // per-story on/off
W.moveTo('Village');             // move the player
W.advanceClock(2);               // pass two time steps
W.setClock({ season: 'autumn' }); // set clock fields
W.giveItem('Torch', 2);          // inventory
W.setFlag('metRowan', true);     // flags
W.setQuest('find_sword', 'active');
W.applyTags('<move>Tavern</move>'); // apply tag text directly
W.preview();                    // the current slice, as a string
W.exportWorld();                 // portable JSON
W.exportWorldAsWI();             // flat WorldInfo array
```

10. Tips & troubleshooting

- **Nothing is injected?** Make sure a world is selected **and** "Enable for this story" is on, and that you've set a current location.
- **AI ignores locations?** Add explicit guidance in **World Rules**, and give locations a short description so they read clearly.
- **A location has no detail** — that's fine; the `[Current Location: ...]` header always appears so the AI still knows where it is.
- **Events not firing** — events trigger on authored conditions (time/season/flags). Basic events can be placed in the editor; detailed conditions/effects are set via imported data or the console for now.
- **Start over for a chat** — just toggle the world off, or switch to a different world in the panel.